PCAP Starting testing

Date	August 7, 2026 → August 10, 2026

Phase 1: Multi-Attacker & Multi-Target Simulation via PCAP Replay

We are capturing real web attack traffic, rewriting its headers to simulate a multi-host network topology and continuously replaying it to test Suricata's detection and feed live dashboard analytics.

This phase uses `tcprewrite` and `tcpreplay` to take a single attack capture (or collection of attacks) and map it across **fake attacker IPs** targeting **fake victim IPs**.

Because Suricata listens in promiscuous mode on interface `ens34`, replaying these modified frames tricks the IDS into seeing a multi-host network without needing 20 separate virtual machines running simultaneously.

PCAP stands for **Packet Capture**, and it refers directly to recorded or captured network traffic.

In computer networking, PCAP refers to two main things:

1. **The File Format (`.pcap` / `.pcapng`):** A standard file format used to save raw network traffic recorded directly off a network interface (NIC).
2. **The Software Library/API:** System libraries (like `libpcap` on Linux or `Npcap` on Windows) that allow security tools to intercept network packets directly from the hardware layer.

What is inside a PCAP file?

When you save a capture, the PCAP file acts like a digital audio/video recording of your network wire. For every single frame captured, it stores:

- **Timestamps:** Exactly down to the microsecond when the packet hit the network card.

- **Layer 2 (Data Link):** MAC addresses (Source and Destination Ethernet addresses).
- **Layer 3 (Network):** IP addresses (Source, Destination, TTL, protocol flags).
- **Layer 4 (Transport):** Ports (TCP/UDP source and destination ports, sequence numbers).
- **Layer 7 (Application Payload):** Unencrypted data contents, such as HTTP requests, DNS queries, TLS handshakes, or raw attack payloads.

Step 1: Installed the `tcpreplay` Suite

```
us@ubuntusur:~$ sudo apt update && sudo apt install -y tcpreplay tshark
[sudo: authenticate] Password:
Hit:1 http://pk.archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://pk.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Hit:3 http://pk.archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:4 http://pk.archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [446 kB]
Get:5 http://pk.archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [111 kB]
Get:6 http://pk.archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [84.8 kB]
Get:7 http://pk.archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [331 kB]
Get:8 http://pk.archive.ubuntu.com/ubuntu resolute-updates/restricted Translation-en [63.0 kB]
Get:9 http://pk.archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [235 kB]
Get:10 http://pk.archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [76.9 kB]
Get:11 http://pk.archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [161 kB]
Get:12 http://pk.archive.ubuntu.com/ubuntu resolute-updates/multiverse amd64 Packages [11.4 kB]
Get:13 http://pk.archive.ubuntu.com/ubuntu resolute-updates/multiverse Translation-en [3,032 B]
Get:14 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:15 https://ppa.launchpadcontent.net/oisf/suricata-stable/ubuntu resolute InRelease
Get:16 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [371 kB]
Get:17 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [89.6 kB]
Get:18 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [32.7 kB]
Get:19 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [321 kB]
Get:20 http://security.ubuntu.com/ubuntu resolute-security/restricted Translation-en [61.2 kB]
Get:21 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.1 kB]
Fetched 2,717 kB in 14s (192 kB/s)
56 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
  tcpreplay  tshark

Installing dependencies:
  libdumbnet1

Summary:
  Upgrading: 0, Installing: 3, Removing: 0, Not Upgrading: 56
  Download size: 534 kB
  Space needed: 2,571 kB / 24.6 GB available
```

Why We Do This

`tcpreplay` includes low-level network utilities (`tcprewrite`, `tcpreplay`, `capinfo`) designed to edit L2–L4 frame headers and inject raw packets directly into network

interface drivers.

What It Does

- `tcprewrite`: Modifies IP addresses, MAC addresses, ports, and recalculates packet checksums on disk before replay.
- `tcpreplay`: Bypasses the OS kernel's network stack and pushes raw Ethernet frames directly out of the physical/virtual network card (`ens34`).
- `tshark`: Provides command-line packet inspection to verify IP address mappings before injecting traffic.

▼ ***Details about ens and suricata:***

VM	NAT-ish interface (not monitored)	Monitored interface (Suricata watches this)
Kali (attacker)	`eth0` → `192.168.23.130`	`eth1` → `192.168.93.130`
Ubuntu-Benign (victim)	`ens33` → `192.168.23.160`	`ens34` → `192.168.93.139`
Ubuntu-Suricata (sensor)	`ens33` → `192.168.23.159`	`ens34` → `192.168.93.138`, MAC `00:0c:29:9b:a0:64`

Step 2: Acquire or Generate a Baseline Attack PCAP

We need an existing PCAP file containing known malicious signatures (such as Nmap scans, Nikto scans, or exploit payloads) so Suricata has traffic to trigger on.

Option A: Capture a Live Scan (Quickest Method)

Ran this in Kali to capture packets:

```
# Kali VM, terminal 1
sudo tcpdump -i eth1 -w sample_attack.pcap
```

In a second terminal on Kali, ran a quick web scan against Ubuntu Benign:

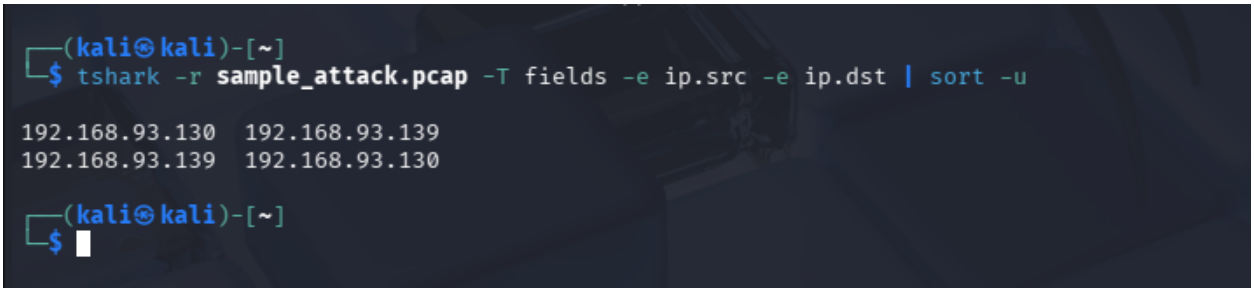
```
# Kali VM, terminal 2
nikto -h http://192.168.93.139/
```

```
# confirm the scan is actually in the capture
tshark -r sample_attack.pcap -Y "http.request" -T fields -e h
ttp.user_agent | sort -u

#result:┌──(kali㉿kali)-[~]
└─$ tshark -r sample_attack.pcap -Y "http.request" -T fields
-e http.user_agent | sort -u
() { :; }; echo 93e4r0-CVE-2014-6271: true;echo;echo;
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/74.0.3729.169 Safari/537.36
This confirms the scan is in the capture.
```

Step 3: Analysis: Inspecting and Decoding Raw PCAP Topology

Inspecting the pcap file to see the IPs:

```
tshark -r sample_attack.pcap -T fields -e ip.src -e ip.dst |
sort -u
```



Detailed Breakdown of the Output

1. `tshark -r sample_attack.pcap`: Opens and parses the binary packet capture file.
2. `T fields -e ip.src -e ip.dst`: Extracts only the Layer 3 Source IP (`ip.src`) and Destination IP (`ip.dst`) headers for every packet, ignoring payload noise.
3. `| sort -u`: Sorts all IP pairs alphabetically and removes duplicates (`u` for unique) to display a clean network map.

Step 4: Rewrite Source & Destination IPs (`tcprewrite`)

Transform the raw capture into **external attacker IPs** targeting **internal victim IPs** on your active `192.168.23.0/24` lab subnet.

Command

Run this command on your Kali Linux terminal:

```
tcprewrite \
  --infile=sample_attack.pcap \
  --outfile=15_vs_5_simulation.pcap \
  --srcipmap=0.0.0.0/0:10.0.0.0/28 \
  --dstipmap=0.0.0.0/0:192.168.93.128/29 \
  --enet-dmac=00:0c:29:9b:a0:64 \
  --fixcsum \
  --fixhdrlen
```

2 IPs in pcap so mapped to 2 IPs:

Issue faced:

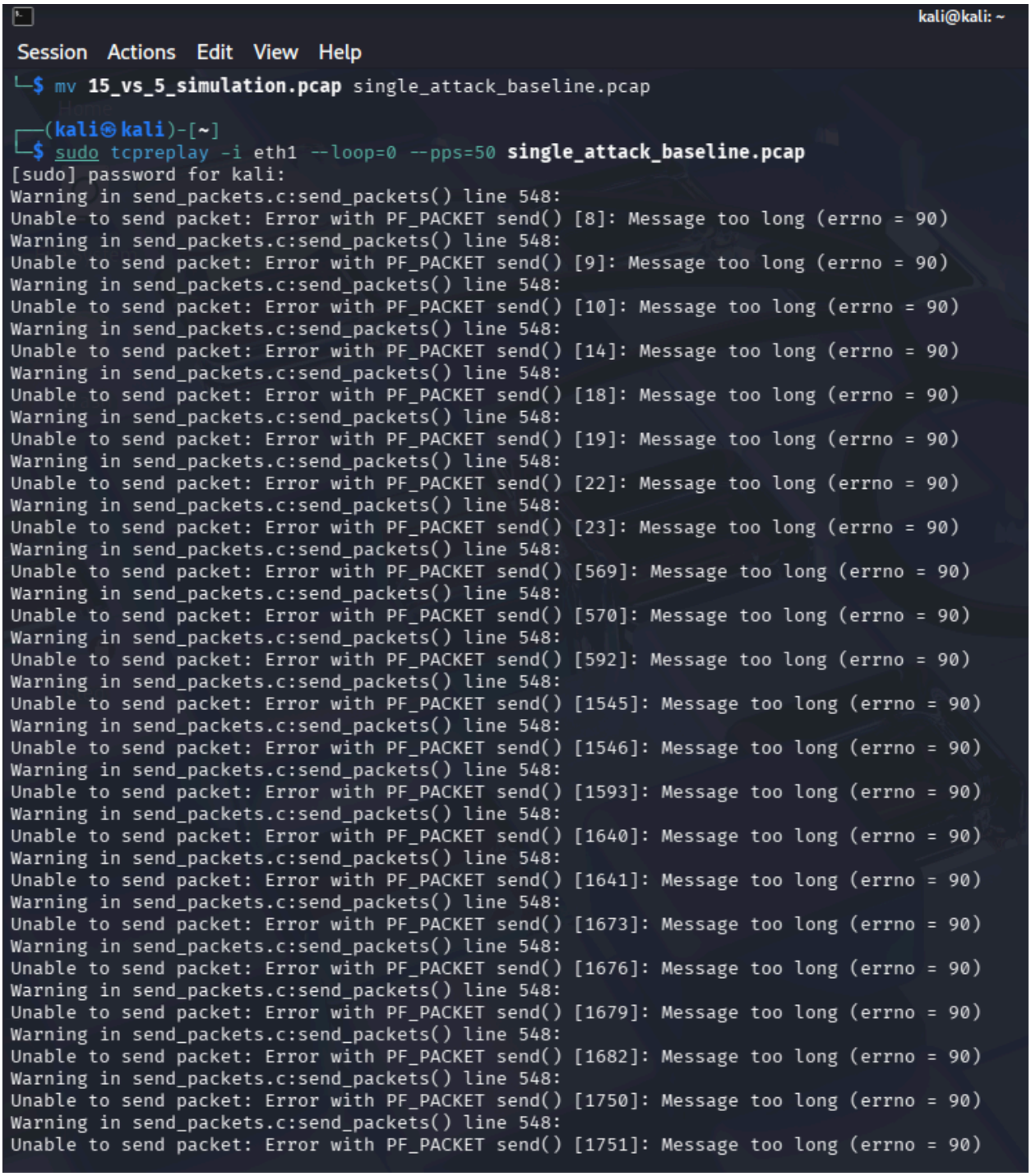
What's happening:

Kali's virtual NIC had TCP/Generic Segmentation Offload enabled, so `tcpdump` captured oversized "super-packets" (way past the 1500-byte MTU) that `tcpreplay` couldn't legally send back out as real Ethernet frames.

`errno 90 = EMSGSIZE ("Message too long")`. This is a classic VMware virtual-NIC artifact, not a problem with your pipeline logic.

Here's the mechanism: your Kali VM's virtual NIC has **TCP Segmentation Offload (TSO)** and related offload features enabled by default. When those are on, the NIC driver lets the kernel hand off *oversized* "super-packets" (sometimes tens of KB) to the network stack, trusting the physical/virtual NIC hardware to split them into proper ≤1500-byte Ethernet frames right before they actually go on the wire. `tcpdump` captures traffic *before* that final splitting happens — so your `sample_attack.pcap` almost certainly has some packets recorded at sizes way beyond the standard 1500-byte MTU.

When `tcpreplay` later tries to blast those same oversized "packets" back out verbatim, the raw socket send() call refuses — a real Ethernet frame physically cannot exceed the interface's MTU, so each oversized packet gets rejected with exactly the error you're seeing.


kali@kali: ~
Session Actions Edit View Help
└─$ mv 15_vs_5_simulation.pcap single_attack_baseline.pcap
┌──(kali㉿kali)-[~]
└─$ sudo tcpreplay -i eth1 --loop=0 --pps=50 single_attack_baseline.pcap
[sudo] password for kali:
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [8]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [9]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [10]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [14]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [18]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [19]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [22]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [23]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [569]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [570]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [592]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [1545]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [1546]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [1593]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [1640]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [1641]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [1673]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [1676]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [1679]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [1682]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [1750]: Message too long (errno = 90)
Warning in send_packets.c:send_packets() line 548:
Unable to send packet: Error with PF_PACKET send() [1751]: Message too long (errno = 90)

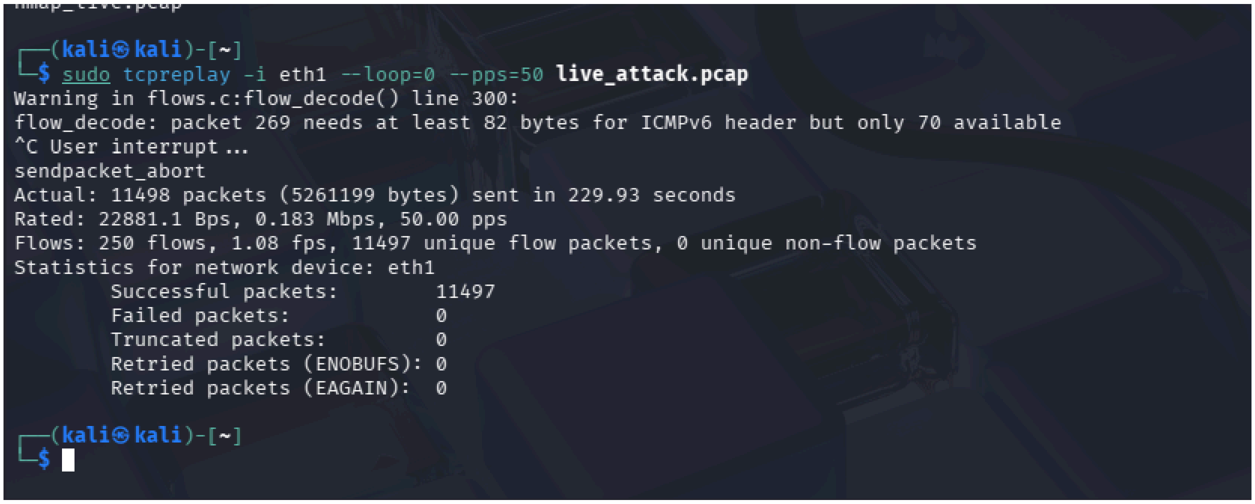
```
┌──(kali㉿kali)-[~]
└─$ sudo ethtool -k eth1 | grep -E "tcp-segmentation|generic-segmentation|generic-receive|large-receive"
tcp-segmentation-offload: on
        tx-tcp-segmentation: on
generic-segmentation-offload: on
generic-receive-offload: on
large-receive-offload: off [fixed]

┌──(kali㉿kali)-[~]
└─$ sudo ethtool -K eth1 tso off gso off gro off lro off

┌──(kali㉿kali)-[~]
└─$ sudo ethtool -k eth1 | grep -E "tcp-segmentation|generic-segmentation|generic-receive|large-receive"
tcp-segmentation-offload: off
        tx-tcp-segmentation: off
generic-segmentation-offload: off
generic-receive-offload: off
large-receive-offload: off [fixed]
```

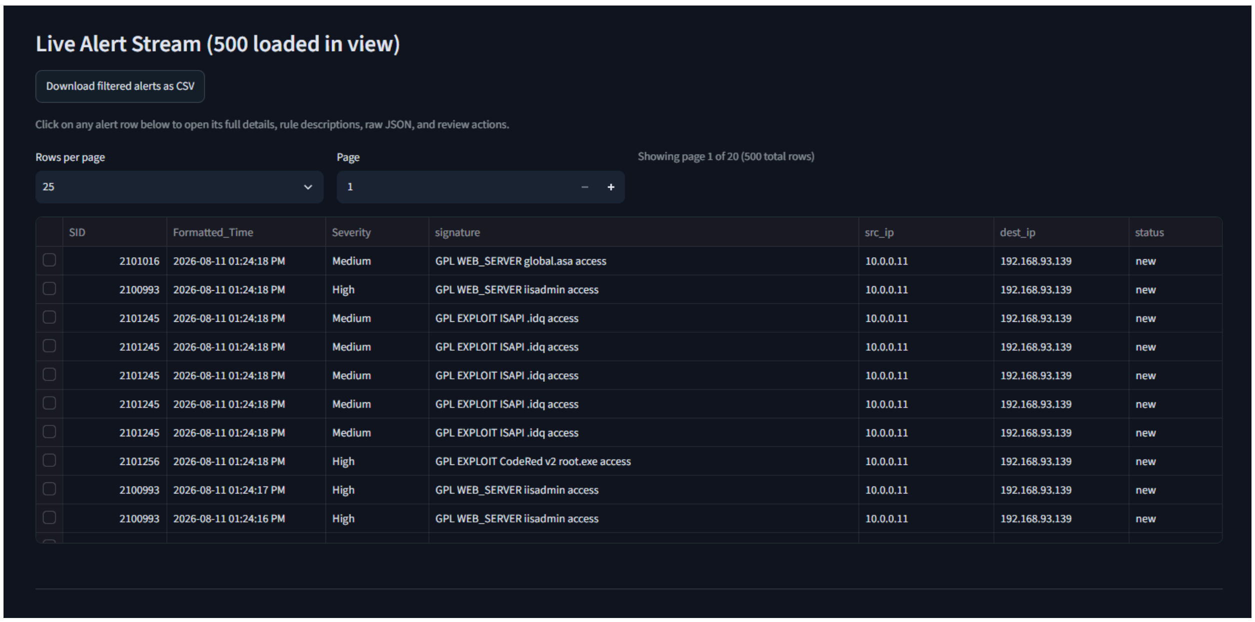
Confirmed output for nikto scan, whether the packet actually contain the nikto capture and now we are ready to replay:

```
┌──(kali㉿kali)-[~]
└─$ tshark -r sample_attack.pcap -Y "http.request" -T fields -e http.user_agent | sort -u
() { :; }; echo 93e4r0-CVE-2014-6271: true;echo;echo;
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/74.0.3729.169 Safari/537.36

┌──(kali㉿kali)-[~]
└─$ sudo tcprewrite \
  --infile=sample_attack.pcap \
  --outfile=single_attack_baseline.pcap \
  --srcipmap=0.0.0.0/0:10.0.0.0/28 \
  --dstipmap=0.0.0.0/0:192.168.93.128/29 \
  --enet-dmac=00:0c:29:9b:a0:64 \
  --fixcsum \
  --fixhdrlen

┌──(kali㉿kali)-[~]
└─$ tshark -r single_attack_baseline.pcap -T fields -e ip.src -e ip.dst | sort -u

10.0.0.11       192.168.93.130
10.0.0.2        192.168.93.131

┌──(kali㉿kali)-[~]
└─$
```

Working Proof:

The dashboard and suricata shows the modified ip alerts now. The replay worked.





SID	Formatted_Time	Severity	signature	src_ip	dest_ip	status
2101016	2026-08-11 01:24:18 PM	Medium	GPL WEB_SERVER global.asa access	10.0.0.11	192.168.93.139	new
2100993	2026-08-11 01:24:18 PM	High	GPL WEB_SERVER iisadmin access	10.0.0.11	192.168.93.139	new
2101245	2026-08-11 01:24:18 PM	Medium	GPL EXPLOIT ISAPI .idq access	10.0.0.11	192.168.93.139	new
2101245	2026-08-11 01:24:18 PM	Medium	GPL EXPLOIT ISAPI .idq access	10.0.0.11	192.168.93.139	new
2101245	2026-08-11 01:24:18 PM	Medium	GPL EXPLOIT ISAPI .idq access	10.0.0.11	192.168.93.139	new
2101245	2026-08-11 01:24:18 PM	Medium	GPL EXPLOIT ISAPI .idq access	10.0.0.11	192.168.93.139	new
2101245	2026-08-11 01:24:18 PM	Medium	GPL EXPLOIT ISAPI .idq access	10.0.0.11	192.168.93.139	new
2101256	2026-08-11 01:24:18 PM	High	GPL EXPLOIT CodeRed v2 root.exe access	10.0.0.11	192.168.93.139	new
2100993	2026-08-11 01:24:17 PM	High	GPL WEB_SERVER iisadmin access	10.0.0.11	192.168.93.139	new
2100993	2026-08-11 01:24:16 PM	High	GPL WEB_SERVER iisadmin access	10.0.0.11	192.168.93.139	new

```
  "sig": "ET WEB_SERVER WEB-PHP phpinfo access",
  "src": "10.0.0.11",
  "dst": "192.168.93.139"
}
{
  "sig": "ET WEB_SERVER Script tag in URI Possible Cross Site Scripting Attempt",
  "src": "10.0.0.11",
  "dst": "192.168.93.139"
}
{
  "sig": "ET WEB_SERVER WEB-PHP phpinfo access",
  "src": "10.0.0.11",
  "dst": "192.168.93.139"
}
{
  "sig": "GPL WEB_SERVER printenv access",
  "src": "10.0.0.11",
  "dst": "192.168.93.139"
}
{
  "sig": "ET WEB_SERVER Script tag in URI Possible Cross Site Scripting Attempt",
  "src": "10.0.0.11",
  "dst": "192.168.93.139"
}
{
  "sig": "ET WEB_SERVER Script tag in URI Possible Cross Site Scripting Attempt",
  "src": "10.0.0.11",
  "dst": "192.168.93.139"
}
{
  "sig": "ET WEB_SERVER Script tag in URI Possible Cross Site Scripting Attempt",
  "src": "10.0.0.11",
  "dst": "192.168.93.139"
}
{
  "sig": "ET WEB_SERVER Script tag in URI Possible Cross Site Scripting Attempt",
  "src": "10.0.0.11",
  "dst": "192.168.93.139"
}
{
  "sig": "ET WEB_SPECIFIC_APPS PHP Classifieds class.phpmailer.php lang_path Parameter Remote File Inclusion Attempt",
  "src": "10.0.0.11",
  "dst": "192.168.93.139"
}
```


Suricata PCAP Replay Lab — Errors & Fixes (Notes)

Goal of this phase

Take a real captured attack (Nikto scan against Ubuntu-Benign), disguise the attacker's IP address, replay it onto the network, and prove Suricata still detects it and the dashboard still shows it. This became the foundation for the bigger 15-vs-5 multi-host simulation.

---

1. Wrong capture interface (found first, before anything else)

What happened: The very first capture used `tcpdump -i eth0`. `eth0` is Kali's NAT-style management interface, not the interface connected to the isolated lab

segment Suricata actually watches. Suricata only listens on `ens34`, which corresponds to Kali's `eth1`.

Symptom: The resulting pcap was full of background noise (gateway traffic, DNS, Ubuntu update servers) but contained zero packets of the actual Nikto scan.

Fix: Always capture on `eth1`, confirmed via `ip a` to be the interface on the `192.168.93.0/24` segment.

Lesson: Which interface you capture on matters as much as what tool you use. Always check `ip a` on both ends and confirm the capture interface matches the monitored segment before trusting a capture.

---

2. Oversized packet errors (`Message too long` / `errno 90`)

What happened: Kali's virtual network interface had TCP Segmentation Offload (TSO) and Generic Segmentation Offload (GSO) turned on by default. This makes the OS record traffic as oversized "super-packets" — bigger than the normal 1500-byte network limit (MTU) — and rely on the network card to split them right before sending. `tcpdump` captured these oversized packets as-is.

Symptom: When replaying the pcap later with `tcpreplay`, the OS refused to send these oversized packets, throwing `errno 90 (Message too long)`.

Fix: Turn off offloading on the capture interface before capturing:

bash

```
sudo ethtool -K eth1 tso off gso off gro off lro off
```

Then recapture. Confirmed fixed once `frame.len` topped out at `1514` bytes (1500 MTU + 14-byte Ethernet header) instead of tens of thousands.

Lesson: Always disable NIC offload features before any packet capture meant for later replay. This is a well-known issue on virtual machines specifically.

---

3. ICMPv6 truncation warnings

What happened: During replay, `tcpreplay` printed a warning like:

```
flow_decode: packet 269 needs at least 82 bytes for ICMPv6 header but only 70 available
```

What it means: Some background IPv6 packets in the original capture were slightly cut short (truncated), so their header didn't have all the expected bytes.

Impact: None on the actual test. This warning is about unrelated background IPv6 traffic, not the attack traffic itself. It doesn't stop the replay and doesn't affect detection.

Lesson: Not every warning is a blocker — confirm what traffic a warning is actually about before spending time chasing it.

---

4. Missing destination MAC address (Layer 2 frame drops)

What happened: Early replay attempts didn't set the destination MAC address on the rewritten packets. VMware's virtual switch uses MAC addresses (not IP addresses) to decide where to deliver frames at the network hardware level.

Symptom: Frames were silently dropped by the virtual switch before ever reaching Suricata's `ens34` interface — even though the IP addresses and payload were otherwise correct.

Fix: Explicitly set the destination MAC to Suricata's real MAC address during the rewrite:

bash

```
--enet-dmac=00:0c:29:9b:a0:64
```

Lesson: IP-level correctness isn't enough for delivery — Layer 2 (MAC address) has to be correct too, especially in virtual network environments.

---

5. Wrong destination subnet / outside HOME_NET

What happened: Early rewrites sent traffic to `192.168.23.128/29` — a different, unmonitored subnet, and one outside Suricata's configured `HOME_NET` (`192.168.93.0/24`).

Symptom: Even if packets had been delivered, many detection rules require the destination to fall inside `HOME_NET` to match. Wrong subnet meant no realistic chance of alerts either way.

Fix: Confirmed `HOME_NET` via `suricata.yaml`, then targeted rewritten destination IPs inside that range.

Lesson: Always check `HOME_NET` before choosing fake IP ranges for a simulation.

---

6. Misunderstanding how `tcprewrite` builds IP mappings

What happened: Assumed that giving `tcprewrite` a wide IP range (like `10.0.0.0/28`) would automatically create many different fake attacker IPs from one capture.

Reality: `tcprewrite` only assigns one fake IP per *unique real IP* it finds in the file. A capture of one attacker talking to one victim only ever has 2 real IPs in it — so no matter how wide the target range is, only 2 fake IPs come out.

Fix: To get many fake attackers, the same original capture has to be rewritten separately, once per fake attacker identity, then merged together with `mergecap`.

Lesson: `tcprewrite` disguises IPs that already exist in the file — it doesn't invent new traffic or new hosts.

---

7. Unidirectional traffic / missing return flows (the flow-split bug)

What happened: Using a wide, "match everything" style rewrite (`0.0.0.0/0`) for both source and destination mapping caused the *same real host* to be given two different, unrelated fake identities — one identity when it appeared as a source, a different identity when it appeared as a destination.

Why this breaks things: Suricata needs to see a matching request and a matching reply — with the IPs correctly reversed — to consider a connection "established." Once the request and reply were rewritten into two different, disconnected identities, Suricata could never match them up as the same conversation. So the connection was never marked established, and most detection rules require an established connection before checking anything else.

Symptom: Alerts stayed at zero, even though the actual attack content was genuinely present in the file.

Fix: Rewrite each specific real IP address individually (using `/32`, meaning "this exact address only") to one specific fake IP, applied consistently in both the source and destination mapping. This way, each real host keeps one single fake identity everywhere it appears in the file, and the request/reply pair stay recognizable as the same conversation.

Lesson: When disguising a real conversation, both sides need to keep one consistent fake identity — not get randomly reassigned depending on which direction they're going in.

---

8. Replaying old/outdated pcap files

What happened: After making fixes and rebuilding a corrected pcap, several replay attempts accidentally used old files (`single_attack_baseline.pcap`, `forward_only.pcap`) left over from earlier attempts, instead of the newly rebuilt one.

Symptom: Continued to see zero alerts, misleadingly suggesting the fix hadn't worked, when really the fix was just never actually tested.

Fix: Rebuilt the pcap fresh with the corrected `/32` rewrite command, verified its contents directly with `tshark` before replaying, and made sure the exact right filename was used going forward.

Lesson: Always double check *which* file is actually being replayed, especially after several rounds of edits — it's easy to accidentally reuse a stale version.

---

9. `tail -f` timing mistake

What happened: `tail -f` only prints new lines added to a file *after* the command starts running — it doesn't show anything that happened earlier. Several times, the replay was started first, and the `tail -f eve.json` monitoring command was started afterward.

Symptom: Since the replay could finish in well under a minute, by the time `tail -f` started watching, the alerts had already been written and there was nothing new left to display — making it look like nothing had happened.

Fix: Always start `tail -f` on the Suricata VM *first*, confirm it's actively running, and only then start the replay on Kali.

Lesson: Order of operations matters when watching a live log — start watching before triggering the thing you're watching for.

---

10. Missing log output folders

What happened: Several offline test commands (`suricata -r ... -l /tmp/some_folder/`) failed immediately because the specified output folder didn't exist yet — Suricata doesn't create it automatically.

Fix: Always `mkdir -p` the target folder before running the command.

---

11. Testing against the wrong / overly specific detection rule

What happened: To confirm detection was working, testing was narrowed down to a single rule at a time. The first rule tried was written specifically for a QNAP device vulnerability, requiring a specific URL (`authLogin.cgi`) that Nikto's traffic never used — so it could never have matched, regardless of anything else being right.

Fix: Wrote a simple custom rule matching exactly the pattern actually present in the captured traffic, to confirm the pipeline itself worked, separate from questions about which official rule should apply.

Lesson: When isolating a test down to one rule, make sure that rule is actually relevant to the traffic being tested — otherwise a "no alert" result doesn't tell you anything useful.

---

12. Files not copied between VMs

What happened: Several `suricata -r` test commands were run on the Suricata VM referencing pcap files that only existed on the Kali VM.

Fix: Used `scp` to copy pcap files from Kali to the Suricata VM before running any tests against them there.

Lesson: Kali is where traffic is captured and rewritten; the Suricata VM is where detection is tested. Files need to be deliberately moved between the two — they don't share storage.